Asymmetric Key Cryptography

Lecture #02

Complexity of computation

- The complexity of an algorithm is a measure of the resources needed to run that algorithm. There could be several types of resources needed.
- The most common resources that are analyzed are
  - Processor time required to run. This is usually expressed as iterations required
  - Memory (Space) required to run
- The complexity is expressed as a function of the size of the input to the algorithm. Given an algorithm *A* its complexity is expressed as *f(n)* where *n* is the size of the input and *f* is some function
- The complexity of a problem is usually understood to mean the complexity of the best known algorithm to solve that problem

Complexity of searching for a number *x* in a list of *N* numbers

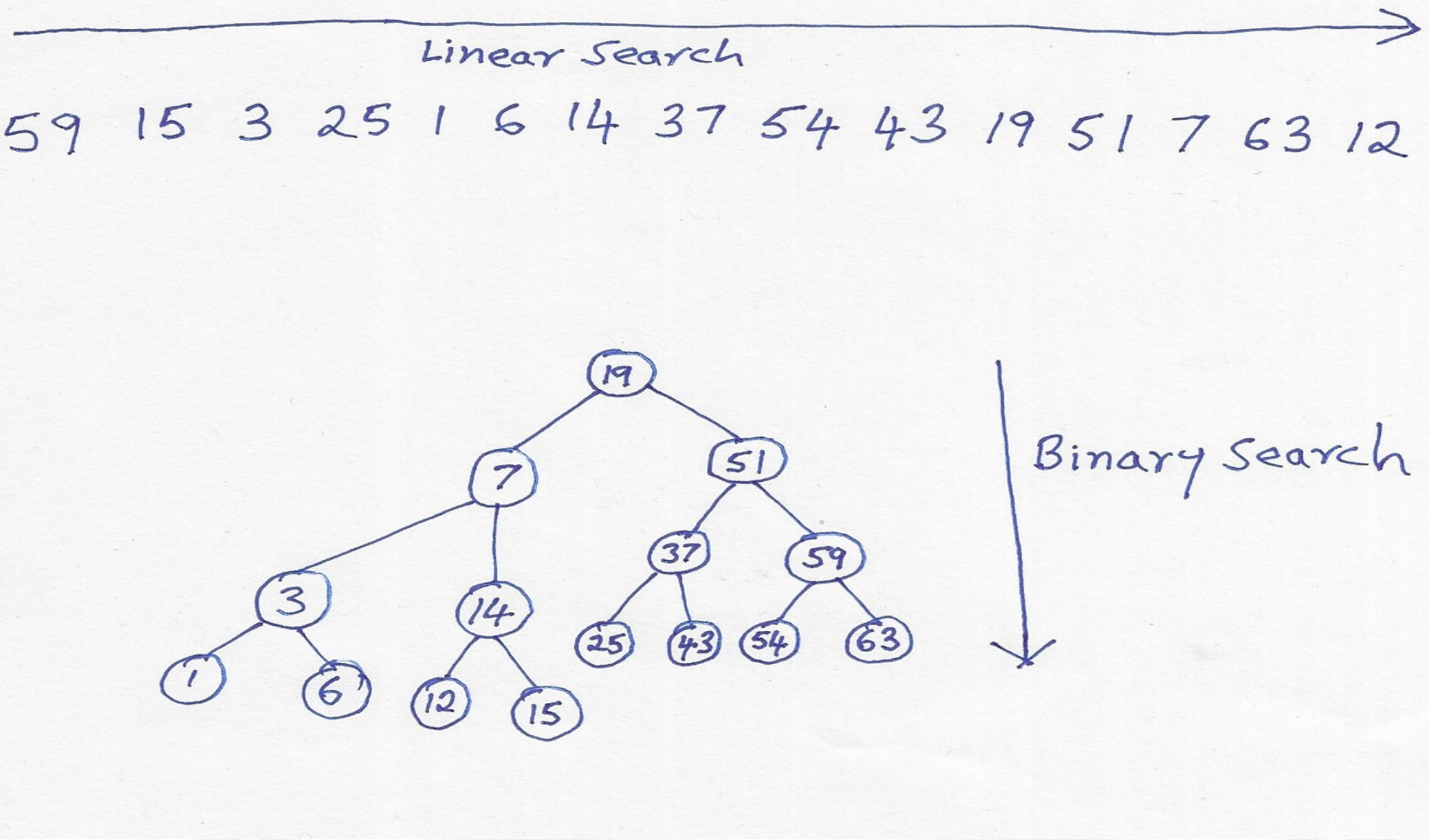
Linear search algorithm:

Check each number in the list one by one starting from the first and progressing till the end. On the average about *N/2* numbers have to be examined. Therefore the complexity of this algorithm is denoted by *O(N)*

Binary tree search algorithm:

If the numbers are sorted in some order (for example in ascending order), then the method used can be to check the number in the middle of the list. If this is equal to x the number has been found. If this is less that x then only the later half is to be searched else the earlier half has to be searched. The complexity of this algorithm is denoted by $O(log_2(N))$

The best known searching algorithm for searching is $O(log_2(N))$ so it is said that the complexity of search is $O(log_2(N))$


Linear Search
59 15 3 25 1 6 14 37 54 43 19 51 7 63 12
19
7
51
3
14
37
59
1
6
12
15
25
43
54
63
Binary Search

Complexity of sorting a list of *N* numbers

Insertion sort algorithm:

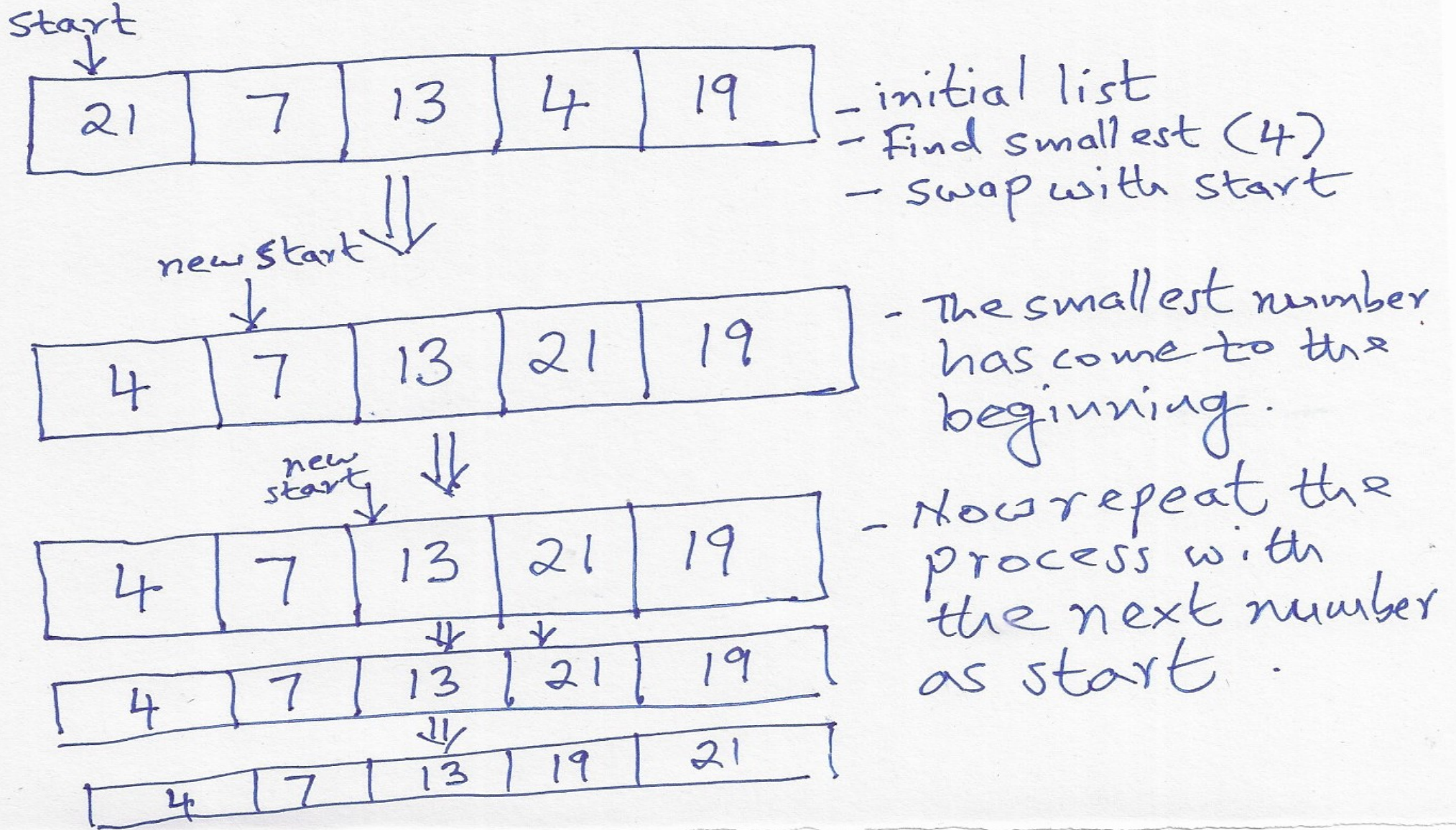
Go through the list and find the smallest number. Swap this with the number in the first position. Then repeat this procedure starting from the second position. Repeatedly do this till the remaining part of the list has only one number left. The complexity of this algorithm is $O(N^2)$ where N is the length of the list of numbers to be sorted

Merge sort algorithm:

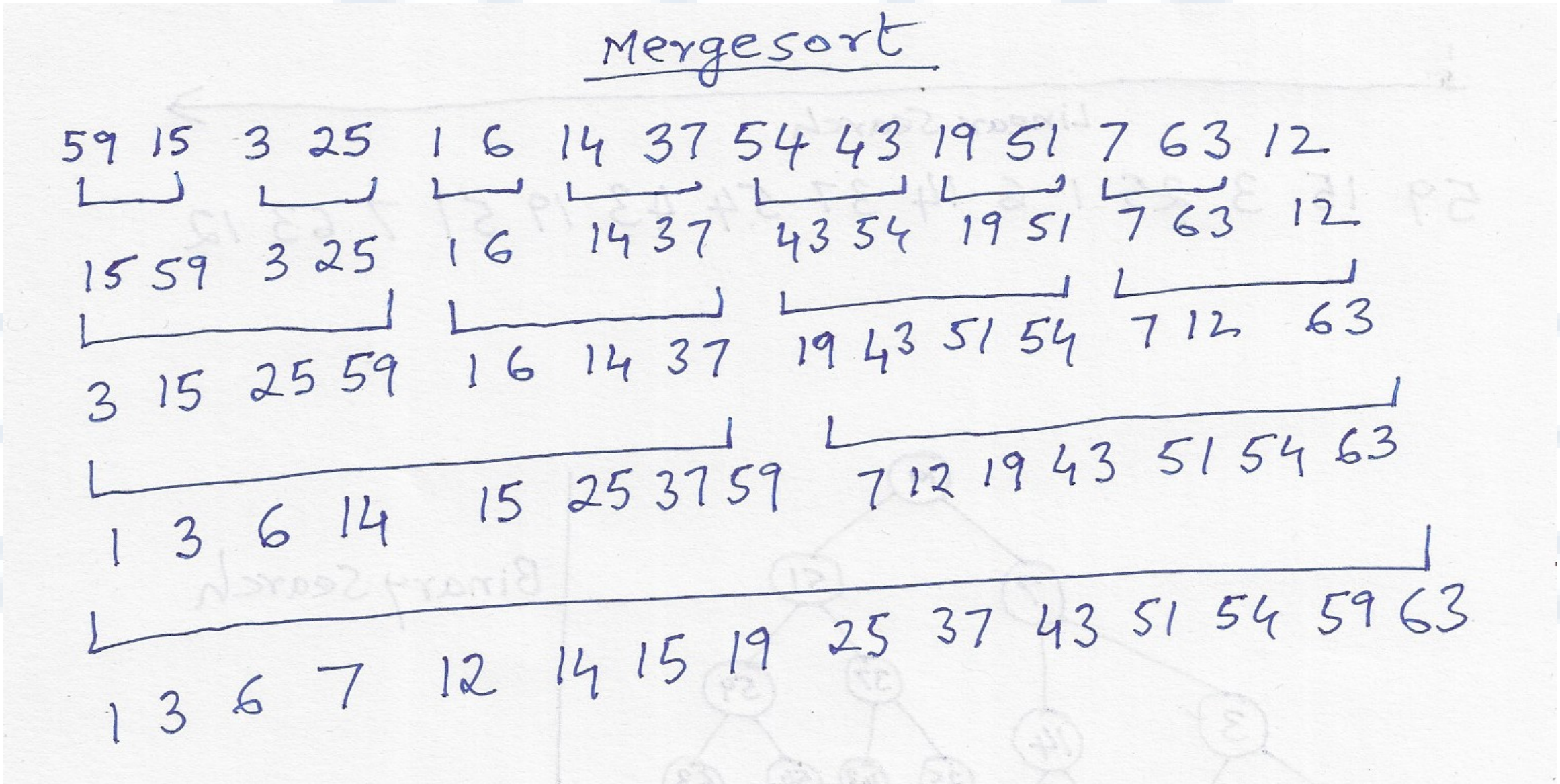
If two sorted lists are available they can be merged into one sorted list in one pass over each list. Thus is we start with lists of length 1 which are trivially sorted and merge adjacent pairs of numbers. Then we take these lists of two numbers each and merge adjacent lists and so on until we have one list left. The complexity of this algorithm is $O(N\log_2(N))$

The best known sorting algorithm for is $O(N\log_2(N))$ so it is said that the complexity of sorting is $O(N\log_2(N))$

INSERTION SORT



Mergesort


59 15 3 25 1 6 14 37 54 43 19 51 7 63 12
15 59 3 25 1 6 14 37 43 54 19 51 7 63 12
3 15 25 59 1 6 14 37 19 43 51 54 7 12 63
1 3 6 14 15 25 37 59 7 12 19 43 51 54 63
1 3 6 7 12 14 15 19 25 37 43 51 54 59 63

Complexity of addition of natural numbers

The standard method of digit by digit addition assumes that the numbers are represented in a positional number system

C_n		c_1	
d_{1n}	……	d_{11}	d_{10}
d_{2n}	……	d_{12}	d_{20}
r_n		r_1	r_0
$c_{(n+1)}$		c_2	c_1

1	0	0	1	1	
6	7	0	3	5	9
2	5	8	2	7	6
9	2	8	5	3	5

This algorithm is linear in the length of the numbers to be added. However if the value of a number in a positional number system is considered, a number of length *n* could have a value of upto R^n where *R* is the radix of the positional number system being used. Therefore an *n* digit binary number can have a value of upto 2^n and an *n* digit decimal number can have a value of upto 10^n

The complexity of addition of natural numbers in binary form is therefore $O(\log_2 N)$ if *N* is taken to be the value of the numbers being added

Exponential Growth

Positional representation

- 1
- 2
- 10
- 100

- 1000

Unitary representation

- I
- II
- IIIIIIIIII
- IIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIII
  IIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIII
  IIIIIIIIIII

  Would take about 3 slides to write

Complexity of multiplication of natural numbers

Multiplication is defined as repeated addition.

Thus $m \times n$ is $\underbrace{m + m + m + m + \ldots\ldots + m}_{n \text{ times}}$

Therefore the complexity of multiplication of two numbers of N binary digits each is $O(\log_2(N)) \times N = O(2\log_2(N)) = O(N.\log_2(N))$

(Note the in O notation we only consider the term of the highest power, i.e. $O(c_o N^0 + c_1 N^1 + \ldots\ldots c_m N^m)$ is considered equivalent to $O(N^m)$)

Hard Problems

- Of the functions 2^N and N^k (where k can be an arbitrarity large positive integer), which has the potential to grow faster as N increases?

$\frac{d(2^N)}{dN} = 2N.log_e 2$ Note that the rate of growth of the function is always greater than the function itself. Every subsequent derivative will also be greater than the previous one.

$\frac{d(N^k)}{dN} = kN^{(k-1)}$ Here the rate of the growth is smaller than the function and the kth derivative will be 0.

Therefore if the best known algorithm for a problem is $O(2^N)$, we say that the problem is a hard problem

Modular Arithmetic

Given two positive integers *i* and *j*

i mod *j* refers to the remainder of dividing *i* by *j*

Example: 13 mod 7 = 6

Given three positive integers *a,b* and *n*

a=b (mod n) if a mod n = b mod n

Example: *16 = 29 mod 13*

Modular arithmetic is a system where all operations are carried out modulo a positive integer. Thus for a modulus of *n*

a + b is computed as *(a + b) mod n*

ab is computed as *ab mod n*

a^b is computed as $a^b mod\ n$

It is generally not true that if $a=b\ mod\ n$ then $k^a\ mod\ n = k^b\ mod\ n$

Example: 3 is a primitive root mod 7

$$
\begin{aligned}
3^1 &= 3^0 \times 3 \equiv 1 \times 3 = 3 \equiv 3 \pmod 7 \\
3^2 &= 3^1 \times 3 \equiv 3 \times 3 = 9 \equiv 2 \pmod 7 \\
3^3 &= 3^2 \times 3 \equiv 2 \times 3 = 6 \equiv 6 \pmod 7 \\
3^4 &= 3^3 \times 3 \equiv 6 \times 3 = 18 \equiv 4 \pmod 7 \\
3^5 &= 3^4 \times 3 \equiv 4 \times 3 = 12 \equiv 5 \pmod 7 \\
3^6 &= 3^5 \times 3 \equiv 5 \times 3 = 15 \equiv 1 \pmod 7
\end{aligned}
$$

The discrete log problem

In a *mod n* system given *a*, *b* and *k* (*a*,*b* and *k* are all less than *n*), *k* is called the discrete logarithm of *b* if $a^k = b$ where all operations are carried out *mod n.*

Given *a*, *b* and *n,* there is no efficient algorithm known that will find a k such that $a^k = b$ where all operations are carried out *mod n.* All known algorithms are some version of keep trying larger and larger values of k and keep checking if a^k has become equal to *b.* Stop when such a *k* has been found.

The integer factorization problem

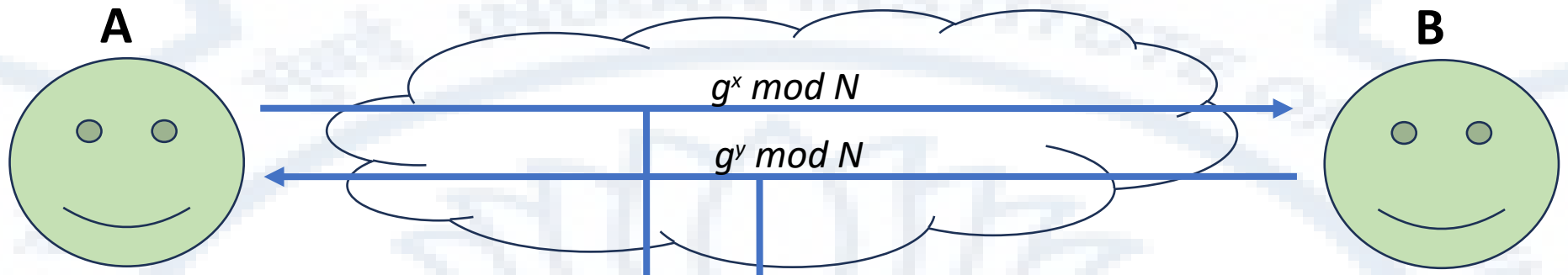
Given an integer N, find its integer factors

Here again there is no efficient way other than start from 2 going up progressively to N ÷ 2 and check to see if the number being considered is indeed a factor of N. There are many refinements but the complexity of all those algorithms remains linear in the value of N which as we have seen is exponential in the size of the representation of N.

The Diffie-Hellman system

- Can two parties A and B communicate over a public channel and carry out some computations such that they are both in possession of a number that cannot be computed by anyone that is intercepting all of their communications?
- The procedure
  - A and B agree on a large *N* which is a prime number and a *g* which is a primitive root modulo *N*
  - A choses a random *x*
  - B choses a random *y*
  - A computes $g^x \ mod \ N$ and sends it to B who uses it to compute $(g^x)^y \ mod \ N = g^{xy} \ mod \ N$
  - B computes g^y and sends it to A who uses it to compute $(g^y)^x \ mod \ N = g^{yx} \ mod \ N = g^{xy} \ mod \ N$
- At the end of the process both A and B know $g^{xy} \ mod \ N$

N and *g* are pre-agreed and publicly known



Choses random *x*
Computes $g^x \bmod N$
Sends $g^x \bmod N$ on an insecure channel
Receives $g^y \bmod N$ on an insecure channel
Computes $(g^y \bmod N)^x = g^{yx} \bmod N$
$= g^{xy} \bmod N$
Now $g^{xy} \bmod N$ can be used as a shared key for a symmetric encryption system.

Eavesdropper intercepts $g^x \bmod N$ and $g^y \bmod N$
Due to the complexity of The discrete log problem
The eavesdropper can not easily compute $g^{xy} \bmod N$

Choses random *y*
Computes $g^y \bmod N$
Sends $g^y \bmod N$ on an insecure channel
Receives $g^x \bmod N$ on an insecure channel
Computes $(g^x \bmod N)^y = g^{xy} \bmod N$

Now $g^{xy} \bmod N$ can be used as a shared key for a symmetric encryption system.

Asymmetric (Public) Key Cryptography

- The Diffie-Hellman system allowed to entities to carry out a procedure between themselves over an insecure channel (where everything sent can be intercepted) and at the end of it have a “shared secret”
- This shared secret could then be used directly or by common agreement be subjected to a process to derive a mutually known key that could then be used to actually encrypt

The RSA (Rivest-Shamir-Adleman) Cryptosystem

Basic principle

- It is practical to find very large numbers *n,e* and *d* such that
  for all m where $0 \leq m < n$
  $(m^e)^d = m \pmod n$

However given only *n* and *e* it is extremely difficult to find (compute) *d*

So if *m* is considered to be a message, then

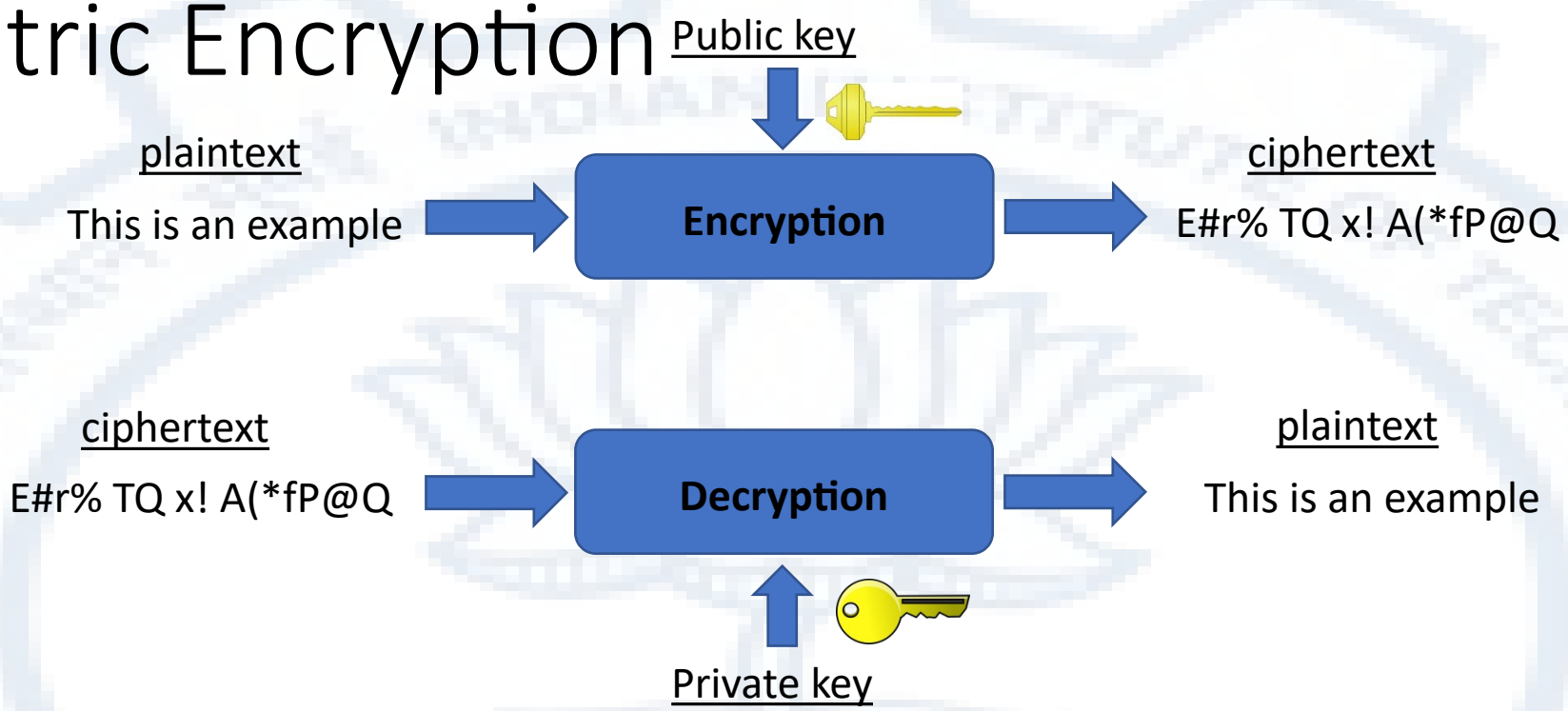
$(m^e) \pmod n$ is encryption resulting in an encrypted message *c* and

$c^d \pmod n$ which is just $(m^e)^d = m \pmod n$ *is* decryption

e is usually called the public key and *d* is called the private key

Note that $(m^e)^d = m^{ed} = (m^d)^e$

Asymmetric Encryption

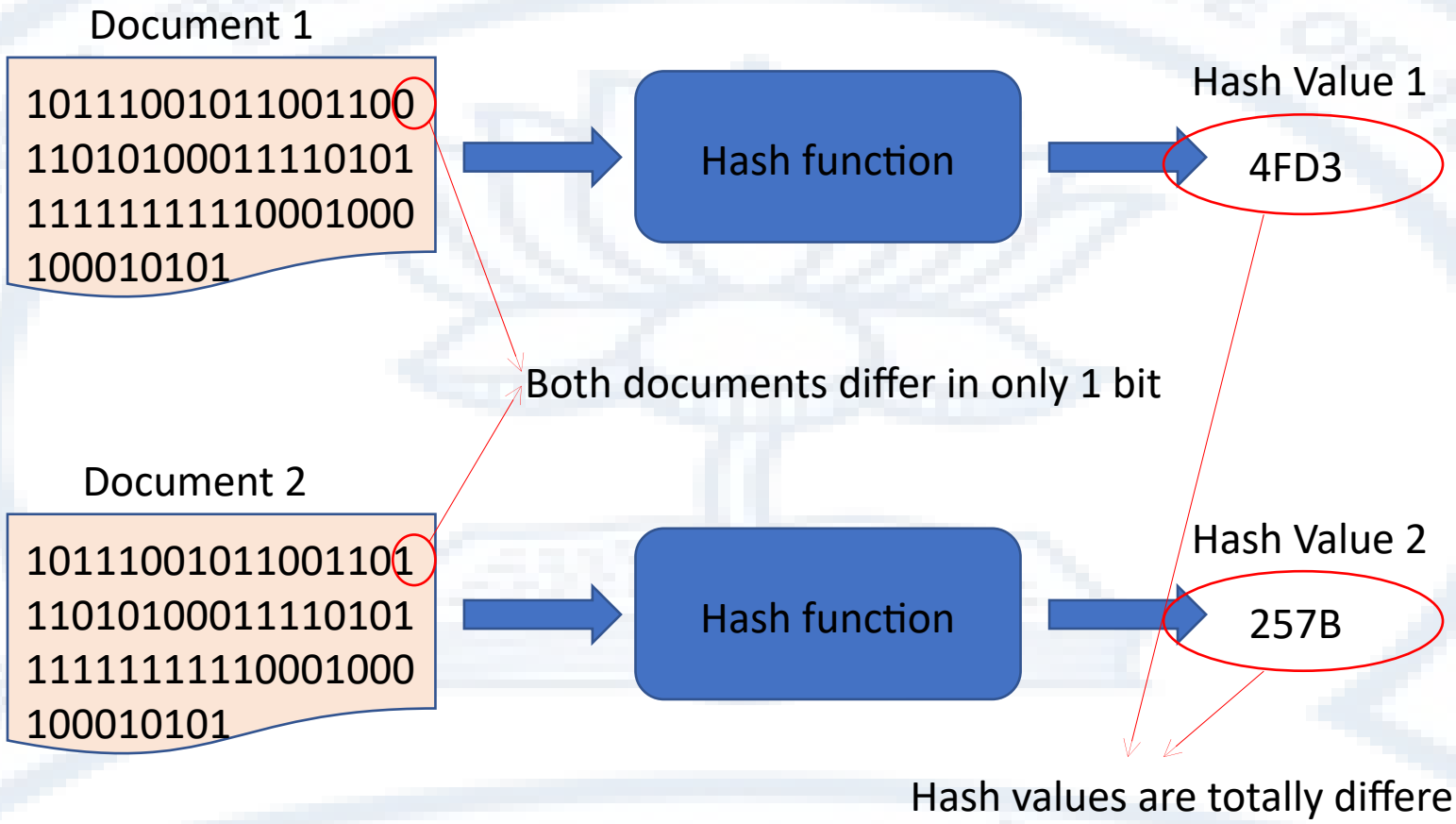


- The key used for encryption (Public key) is different from but mathematically related to the key used for decryption (Private key) and hence the term “asymmetric” key encryption.
- The size of the ciphertext corresponding to a given plaintext is usually significantly bigger than the size of the plaintext.
- A good “cipher” or encryption algorithm is one where security lies only in knowledge of the key. Knowledge of the algorithm should confer no advantage in decryption. Under this assumption the only way to decrypt without a key is to try all possible keys.

Hash Functions

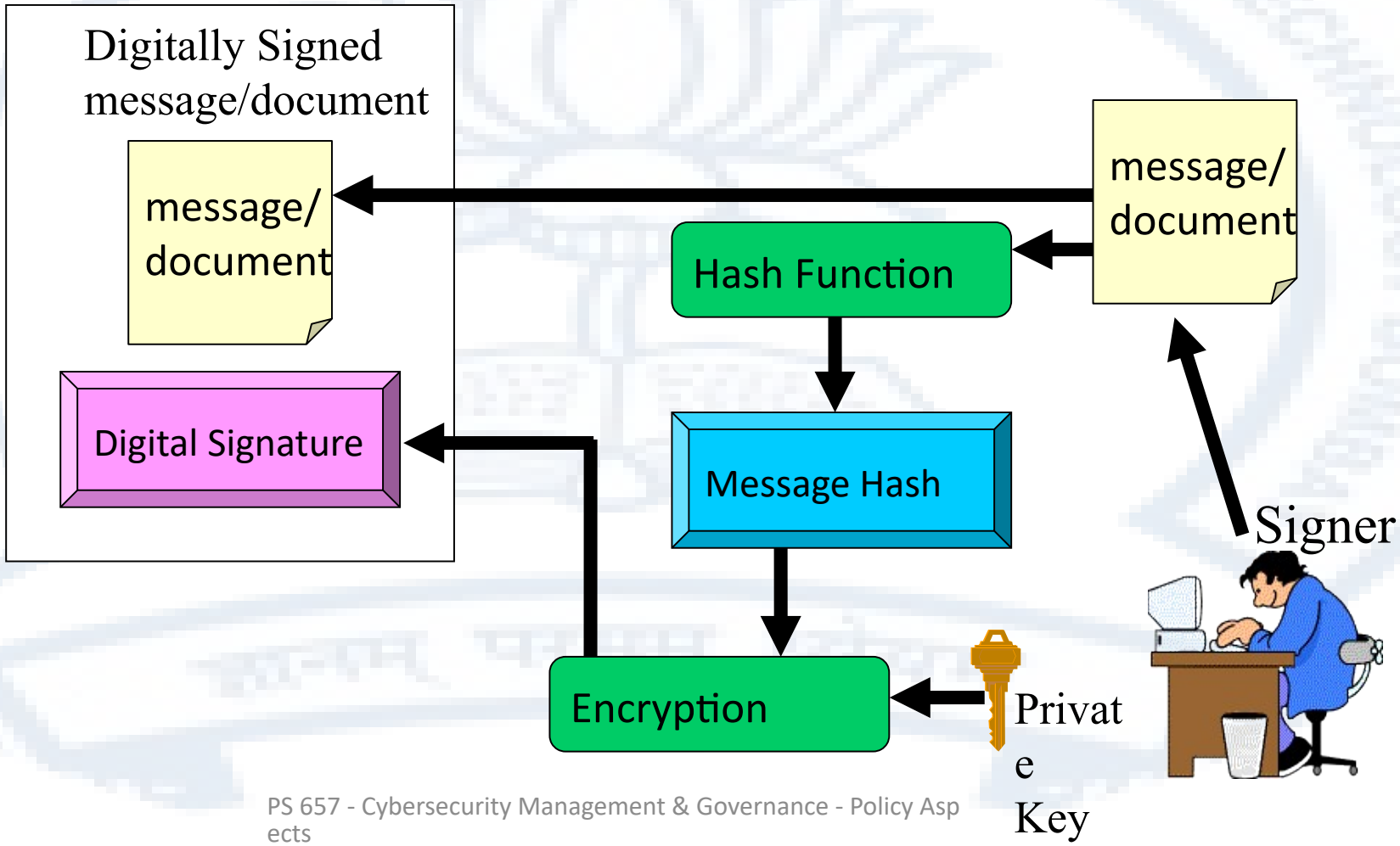
- A hash function takes in an arbitrary input and produces a (usually) small fixed size output
  - An example of a (bad) hash function is the number of vowels in a document
- A hash function can be said to “summarize” a document
- A good hash function will produce uncorrelated hash values for correlated documents
- Since the set of all possible hash values is much smaller than the set of all possible documents, it is possible that many distinct document may have the same hash value. This is called a collision.
- It is extremely unlikely that two meaningfully correlated documents have a collision. Therefore it is safe to use a well designed hash function as a summary for a document
- SHA-256 is an example of a modern well defined hash function

Hash Functions

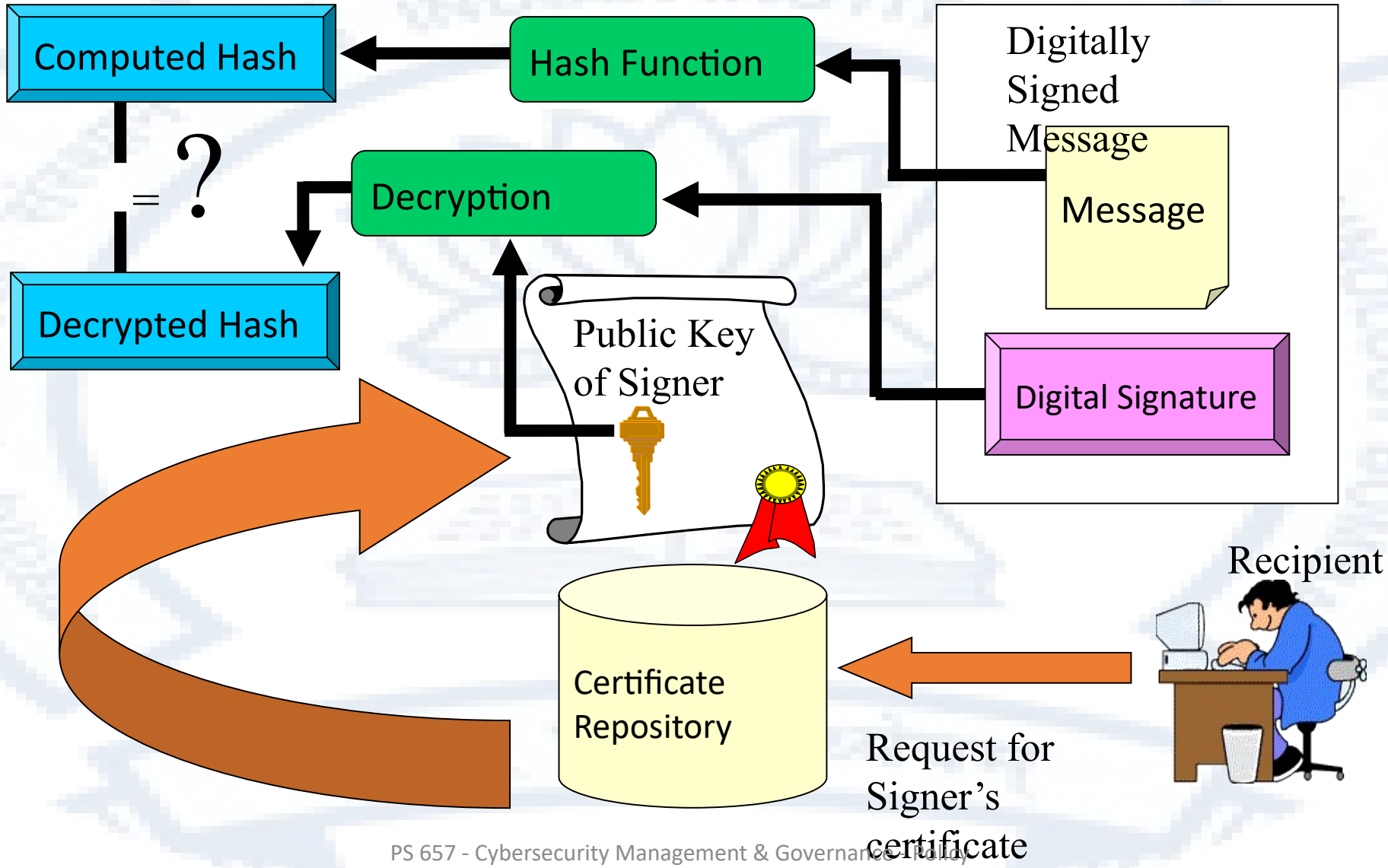


Modern, well designed hash functions ensure that a change in the documents cannot be correlated to a change in the hash values. Even a single bit being different can produces radically different hash values

Digitally signing a message/document



Verifying a digital signature



The impact of Quantum Computing

- In 1994 Peter Shor described an algorithm for factorizing an integer into it's prime factors in *polynomial time* on a quantum computer
- The implication is that if quantum computers become practical, the RSA asymmetric key algorithm will be "broken". While this is a theoretically correct statement, in practical terms we are far away from seeing a general compromise of RSA since
  - the current capabilities of existing quantum computers are not very powerful and doubling the existing key sizes will provide significant protection even for polynomial enumeration
  - The current practice is to encrypt a message-encryption-key with an asymmetric key algorithm like RSA and encrypt the actual content with a symmetric algorithm encrypted with the message-encryption key
  - Most current symmetric key algorithms are not impacted by quantum computing
- There is a ongoing work on quantum resistant and quantum safe cryptography, but there is no practical threat to current cryptosystems from the currently known body of work in Quantum computing

To summarize

- When the number of participants becomes large, it becomes impractical to use a symmetric key cryptosystem since the number of distinct keys becomes too large
- There was a need to be able to negotiate a key over untrusted communication channels.
- The first practical scheme was the Diffie-Hellman scheme and this led to the idea of asymmetric or public key cryptography
- The Diffie-Hellman scheme was quickly followed by the RSA scheme where encryption was done with a public key and decryption with a private key
- The RSA scheme also enables the concept of “digital signature”
- The RSA is theoretically vulnerable to quantum computing attacks however in practical terms increasing keys sizes will work for a few decades

End of

Lecture #2